

Logistic Regression Evaluation

Airline Customer Satisfaction Prediction

Project Overview

In this project, I will use Logistic Regression to predict whether an airline passenger is satisfied or dissatisfied based on survey responses and travel-related factors.

The goal is to identify the key factors that influence customer satisfaction and evaluate how well a Logistic Regression model can classify passengers into satisfaction categories.

Throughout this analysis, I will:

- Explore and clean the dataset
- Encode categorical variables
- Split the data into training and testing sets
- Build a Logistic Regression model
- Evaluate model performance using a Confusion Matrix, Precision, Recall, and Accuracy
- Interpret the model results
- Provide business recommendations based on the findings

Importing Required Libraries

To begin the analysis, I will import the Python libraries needed for data manipulation, visualization, machine learning, and model evaluation.

These libraries will help me prepare the data, build the Logistic Regression model, and assess its performance.

```
In [97]: import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import (
    confusion_matrix,
    classification_report,
    precision_score,
    recall_score,
```

```
accuracy_score
)

import warnings
warnings.filterwarnings("ignore")
```

Loading the Dataset

The first step is to load the airline customer satisfaction dataset into Python.

After loading the dataset, I will inspect the first few rows to understand its structure and verify that the data has been imported correctly.

```
In [98]: df = pd.read_csv("Invistico_Airline.csv")

df.head()
```

Out[98]:

	satisfaction	Customer Type	Age	Type of Travel	Class	Flight Distance	Seat comfort	Departure/Arrival time convenient	F
0	satisfied	Loyal Customer	65	Personal Travel	Eco	265	0	0	
1	satisfied	Loyal Customer	47	Personal Travel	Business	2464	0	0	
2	satisfied	Loyal Customer	15	Personal Travel	Eco	2138	0	0	
3	satisfied	Loyal Customer	60	Personal Travel	Eco	623	0	0	
4	satisfied	Loyal Customer	70	Personal Travel	Eco	354	0	0	

5 rows × 22 columns



Initial Data Exploration

Before building a machine learning model, it is important to understand the dataset.

In this section, I will examine:

- The number of rows and columns
- The available variables
- The data types of each feature

This helps identify any data preparation steps that may be required.

```
In [99]: print("Shape:", df.shape)

print("\nColumns:\n")
print(df.columns)

print("\nData Types:\n")
print(df.dtypes)
```

Shape: (129880, 22)

Columns:

```
Index(['satisfaction', 'Customer Type', 'Age', 'Type of Travel', 'Class',
      'Flight Distance', 'Seat comfort', 'Departure/Arrival time convenient',
      'Food and drink', 'Gate location', 'Inflight wifi service',
      'Inflight entertainment', 'Online support', 'Ease of Online booking',
      'On-board service', 'Leg room service', 'Baggage handling',
      'Checkin service', 'Cleanliness', 'Online boarding',
      'Departure Delay in Minutes', 'Arrival Delay in Minutes'],
      dtype='str')
```

Data Types:

```
satisfaction          str
Customer Type         str
Age                   int64
Type of Travel        str
Class                 str
Flight Distance        int64
Seat comfort          int64
Departure/Arrival time convenient  int64
Food and drink        int64
Gate location         int64
Inflight wifi service int64
Inflight entertainment int64
Online support        int64
Ease of Online booking int64
On-board service      int64
Leg room service      int64
Baggage handling      int64
Checkin service       int64
Cleanliness           int64
Online boarding       int64
Departure Delay in Minutes  int64
Arrival Delay in Minutes  float64
dtype: object
```

Checking for Missing Values

Missing values can affect model performance and may require cleaning before training the model.

I will check each column to determine whether any observations are missing and decide on an appropriate treatment strategy if necessary.

```
In [100... df.isnull().sum()
```

```
Out[100... satisfaction          0
Customer Type          0
Age                    0
Type of Travel         0
Class                  0
Flight Distance        0
Seat comfort           0
Departure/Arrival time convenient  0
Food and drink         0
Gate location          0
Inflight wifi service  0
Inflight entertainment  0
Online support         0
Ease of Online booking  0
On-board service       0
Leg room service       0
Baggage handling       0
Checkin service        0
Cleanliness            0
Online boarding        0
Departure Delay in Minutes  0
Arrival Delay in Minutes 393
dtype: int64
```

Exploring the Target Variable

The target variable for this project is passenger satisfaction.

Understanding the distribution of satisfied and dissatisfied passengers is important because highly imbalanced classes can affect classification performance.

I will examine the frequency of each satisfaction category and visualize the distribution.

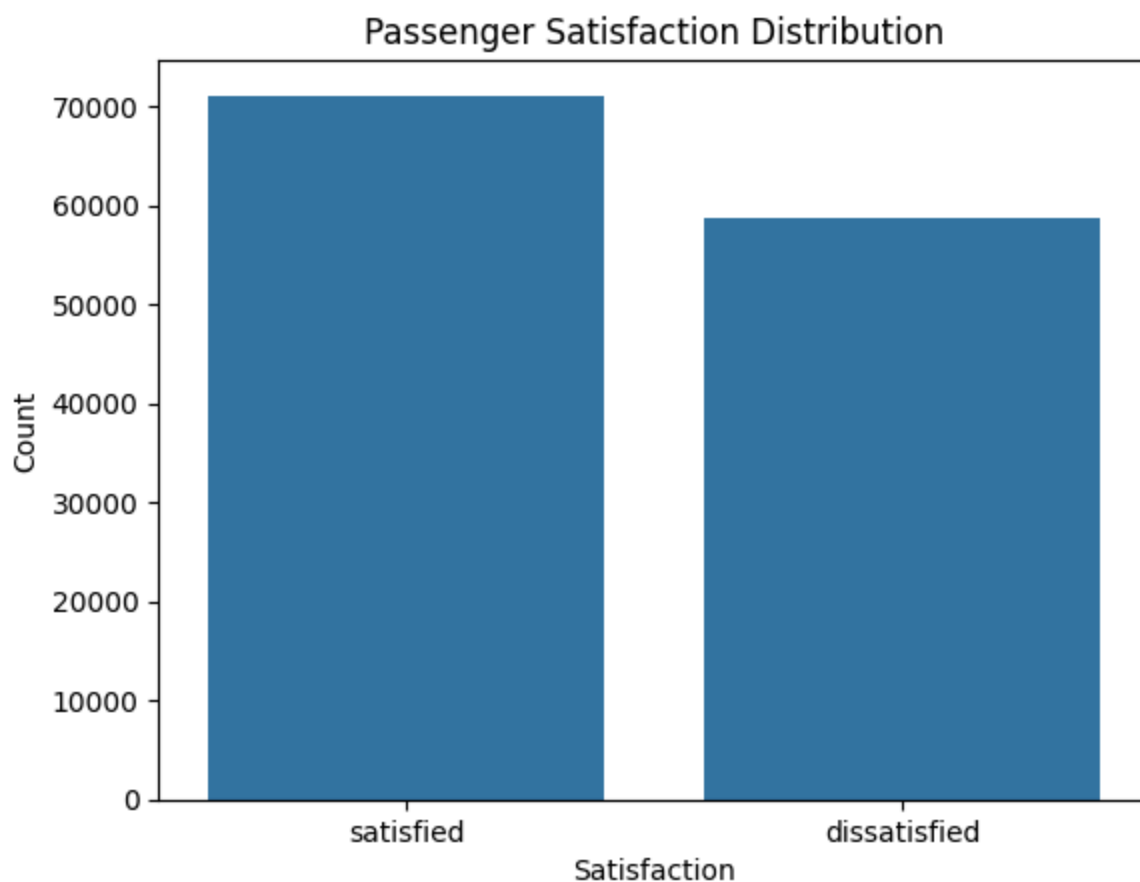
```
In [101... df["satisfaction"].value_counts()
```

```
Out[101... satisfaction
satisfied      71087
dissatisfied    58793
Name: count, dtype: int64
```

```
In [102... sns.countplot(
    data=df,
    x="satisfaction"
)

plt.title("Passenger Satisfaction Distribution")
plt.xlabel("Satisfaction")
plt.ylabel("Count")

plt.show()
```



Target Variable Analysis

The target variable for this project is passenger satisfaction.

The dataset contains 71,087 satisfied passengers and 58,793 dissatisfied passengers. The classes are relatively balanced, which is beneficial for classification modeling because the model will have sufficient examples from both categories during training.

Since there is no severe class imbalance, standard Logistic Regression can be applied without additional resampling techniques.

Handling Missing Values

Before building the Logistic Regression model, it is important to identify and handle any missing values.

Missing values can negatively affect model training and evaluation. In this step, I will determine whether any variables contain missing observations and decide how to address them.

```
In [103... df.isnull().sum().sort_values(ascending=False)
```

```
Out[103...  Arrival Delay in Minutes      393
            satisfaction          0
            Age                  0
            Type of Travel        0
            Class                 0
            Customer Type         0
            Flight Distance        0
            Seat comfort           0
            Food and drink         0
            Departure/Arrival time convenient  0
            Inflight wifi service  0
            Inflight entertainment  0
            Online support         0
            Gate location          0
            Ease of Online booking  0
            On-board service       0
            Baggage handling       0
            Leg room service       0
            Checkin service        0
            Cleanliness            0
            Online boarding        0
            Departure Delay in Minutes  0
            dtype: int64
```

```
In [104... # Total missing values

df.isnull().sum().sum()
```

```
Out[104... np.int64(393)
```

Examining Data Types

Machine learning models require numerical inputs. Therefore, I will inspect the data types of all variables to determine which features need to be encoded before training the model.

```
In [105... df.dtypes
```

```
Out[105...] satisfaction      str
Customer Type      str
Age                int64
Type of Travel     str
Class             str
Flight Distance    int64
Seat comfort       int64
Departure/Arrival time convenient int64
Food and drink     int64
Gate location      int64
Inflight wifi service int64
Inflight entertainment int64
Online support     int64
Ease of Online booking int64
On-board service   int64
Leg room service   int64
Baggage handling   int64
Checkin service    int64
Cleanliness        int64
Online boarding     int64
Departure Delay in Minutes int64
Arrival Delay in Minutes float64
dtype: object
```

```
In [106...] df.select_dtypes(include="object").columns
```

```
Out[106...] Index(['satisfaction', 'Customer Type', 'Type of Travel', 'Class'], dtype='str')
```

Encoding the Target Variable

The satisfaction column contains text labels. Since Logistic Regression requires numerical values, I will convert:

- Satisfied → 1
- Dissatisfied → 0

This transformation allows the model to predict the probability that a passenger is satisfied.

```
In [107...] df["satisfaction"] = df["satisfaction"].map({
    "satisfied": 1,
    "dissatisfied": 0
})

df["satisfaction"].value_counts()
```

```
Out[107...] satisfaction
1      71087
0      58793
Name: count, dtype: int64
```

Encoding Categorical Variables

Several variables in the dataset contain categorical information such as customer type, travel class, and travel purpose.

Since Logistic Regression requires numerical inputs, I will convert these categorical variables into dummy variables using one-hot encoding.

```
In [108... # Identify categorical columns

categorical_cols = df.select_dtypes(include=["object"]).columns

print(categorical_cols)
```

```
Index(['Customer Type', 'Type of Travel', 'Class'], dtype='str')
```

Handling Missing Values

The dataset contains a small number of missing values in the "Arrival Delay in Minutes" variable.

Since the number of missing observations is very small relative to the size of the dataset, I will replace the missing values with the median value of the column. Using the median helps reduce the influence of extreme values and preserves the overall distribution of the data.

```
In [109... df["Arrival Delay in Minutes"] = df["Arrival Delay in Minutes"].fillna(
    df["Arrival Delay in Minutes"].median()
)

# Verify missing values were removed
df.isnull().sum().sum()
```

```
Out[109... np.int64(0)
```

Encoding Categorical Variables

Machine learning algorithms require numerical input variables.

To prepare the dataset for Logistic Regression, I will convert the categorical variables into dummy variables using one-hot encoding.

The first category in each variable will be dropped to avoid multicollinearity and the dummy variable trap.

```
In [110... df_encoded = pd.get_dummies(
    df,
    columns=[
        "Customer Type",
        "Type of Travel",
        "Class"
    ],
```



```
drop_first=True
)
```

```
In [111... print("Original Shape:", df.shape)
print("Encoded Shape:", df_encoded.shape)
```

Original Shape: (129880, 22)
Encoded Shape: (129880, 23)

```
In [112... df_encoded.head()
```

```
Out[112... 
```

	satisfaction	Age	Flight Distance	Seat comfort	Departure/Arrival time convenient	Food and drink	Gate location	Inflight wifi service	entertainment
0	1	65	265	0	0	0	2	2	
1	1	47	2464	0	0	0	3	0	
2	1	15	2138	0	0	0	3	2	
3	1	60	623	0	0	0	3	3	
4	1	70	354	0	0	0	3	4	

5 rows × 23 columns



Defining Features and Target Variable

The target variable is passenger satisfaction.

All remaining variables will be used as predictor variables to help classify whether a passenger is satisfied or dissatisfied.

```
In [113... X = df_encoded.drop("satisfaction", axis=1)
y = df_encoded["satisfaction"]
```

```
In [114... print(X.shape)
print(y.shape)
```

(129880, 22)
(129880,)

Splitting the Dataset

To evaluate model performance fairly, the dataset will be divided into training and testing sets.

The training set will be used to fit the Logistic Regression model, while the testing set will be used to assess how well the model performs on unseen data.

```
In [115... X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42  
)
```

```
In [116... print("Training Set:", X_train.shape)  
print("Testing Set:", X_test.shape)
```

Training Set: (103904, 22)

Testing Set: (25976, 22)

Building the Logistic Regression Model

Logistic Regression is a supervised classification algorithm used to predict the probability of a binary outcome.

In this project, the model will estimate the probability that a passenger is satisfied based on airline service and travel-related features.

```
In [117... log_model = LogisticRegression(  
    max_iter=5000  
)  
  
log_model.fit(  
    X_train,  
    y_train  
)
```

```
Out[117... ▼ LogisticRegression ⓘ ?  
    ► Parameters  
    ► Fitted attributes
```

```
In [118... y_pred = log_model.predict(X_test)  
  
y_pred[:10]
```

```
Out[118... array([1, 0, 1, 1, 0, 1, 0, 1, 1, 0])
```

Model Evaluation

To evaluate the classification model, I will examine:

- Accuracy

- Precision
- Recall
- Confusion Matrix

Precision measures how many predicted satisfied passengers were actually satisfied.

Recall measures how many satisfied passengers were correctly identified by the model.

These metrics provide a more complete assessment than accuracy alone.

```
In [119... accuracy = accuracy_score(y_test, y_pred)

precision = precision_score(y_test, y_pred)

recall = recall_score(y_test, y_pred)

print("Accuracy:", round(accuracy,4))
print("Precision:", round(precision,4))
print("Recall:", round(recall,4))
```

```
Accuracy: 0.8274
Precision: 0.8434
Recall: 0.8429
```

```
In [120... cm = confusion_matrix(y_test, y_pred)

cm
```

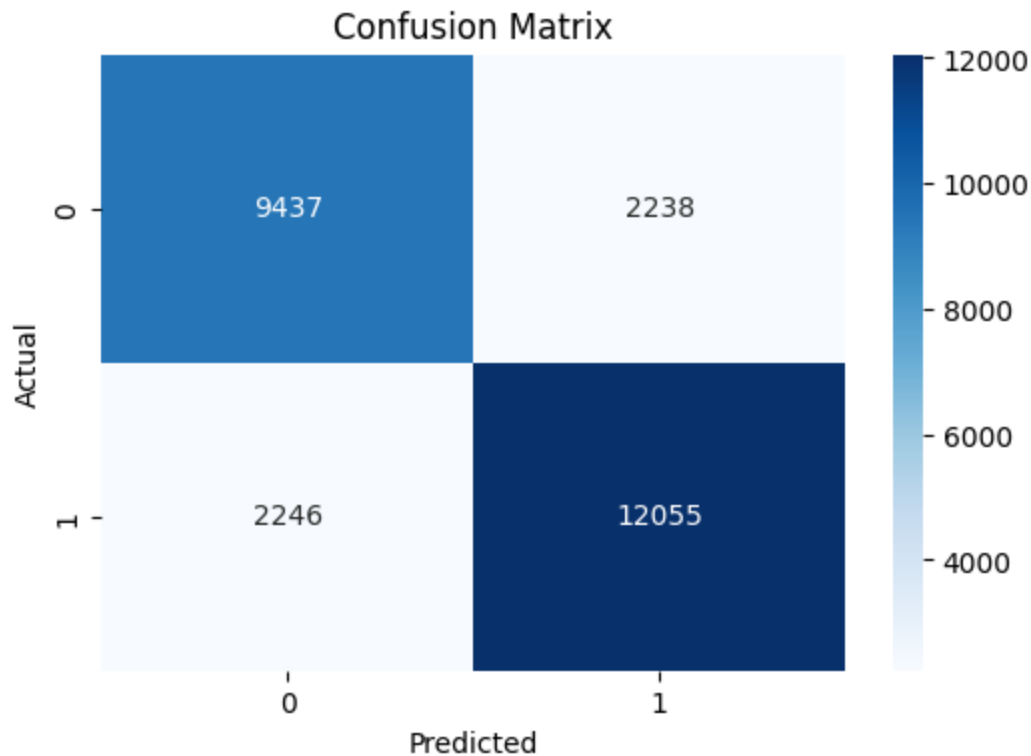
```
Out[120... array([[ 9437,  2238],
        [ 2246, 12055]])
```

```
In [121... plt.figure(figsize=(6,4))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```



Feature Importance Plot

In [122...

```
# Coefficients
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': log_model.coef_[0]
})

# Sort
coefficients = coefficients.sort_values(by='Coefficient')

# Plot
plt.figure(figsize=(10,8))

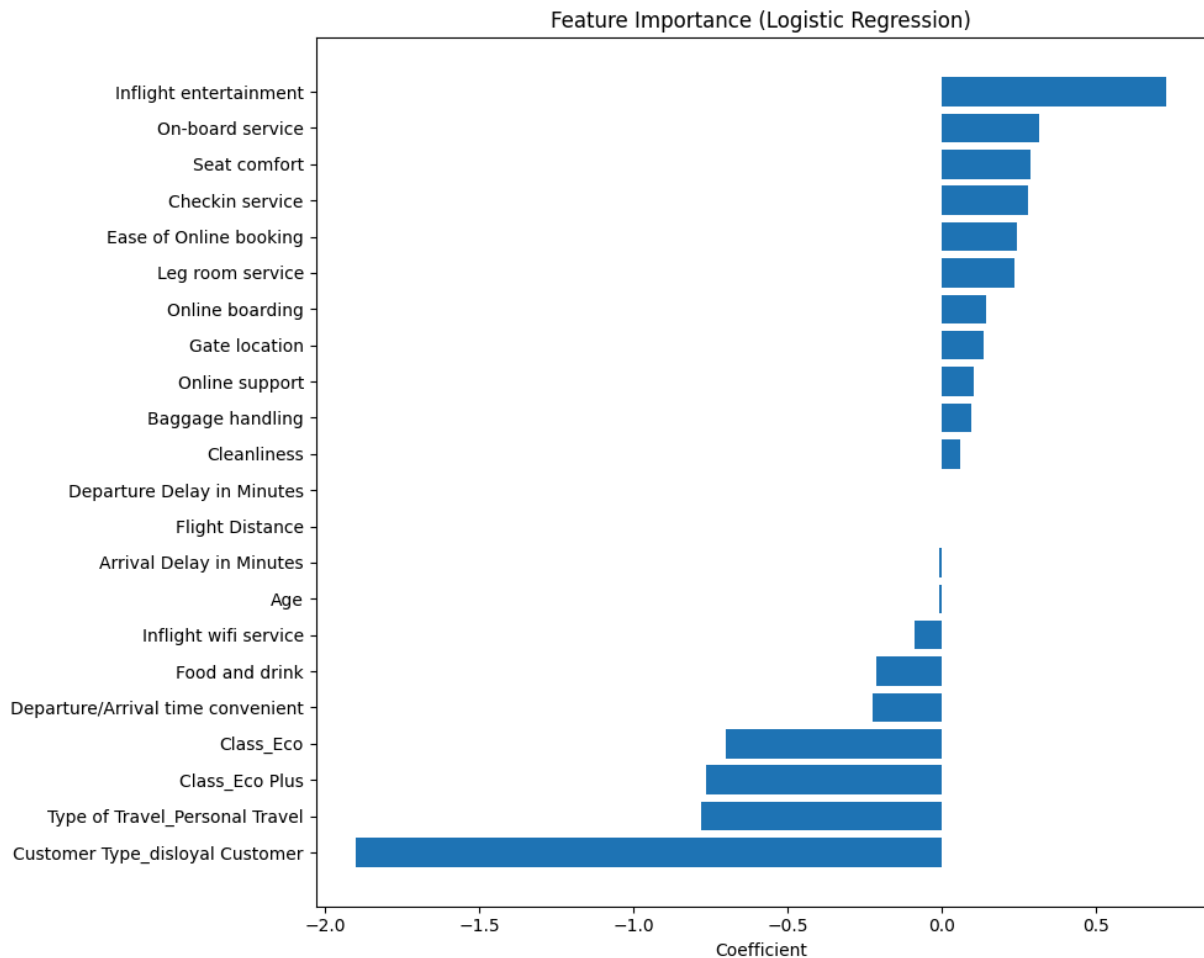
plt.barh(
    coefficients['Feature'],
    coefficients['Coefficient']
)

plt.xlabel('Coefficient')
plt.title('Feature Importance (Logistic Regression)')

plt.tight_layout()

plt.savefig('coefficient_plot.png')

plt.show()
```



```
In [123... print("Accuracy:", round(accuracy,4))
print("Precision:", round(precision,4))
print("Recall:", round(recall,4))
```

Accuracy: 0.8274
Precision: 0.8434
Recall: 0.8429

```
In [124... print(cm)
```

```
[[ 9437  2238]
 [ 2246 12055]]
```

Classification Report

In addition to Accuracy, Precision, and Recall, I will generate a classification report to obtain a detailed summary of model performance for each class.

This report helps evaluate how effectively the model identifies satisfied and dissatisfied passengers.

```
In [125... print(classification_report(y_test, y_pred))
```

	precision	recall	f1-score	support
0	0.81	0.81	0.81	11675
1	0.84	0.84	0.84	14301
accuracy			0.83	25976
macro avg	0.83	0.83	0.83	25976
weighted avg	0.83	0.83	0.83	25976

Identifying Key Drivers of Passenger Satisfaction

Logistic Regression assigns a coefficient to each predictor variable.

Positive coefficients increase the probability that a passenger is satisfied, while negative coefficients decrease that probability.

By examining the largest coefficients, I can identify which service factors have the strongest influence on customer satisfaction.

In [126...

```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": log_model.coef_[0]
})

coefficients = coefficients.sort_values(
    by="Coefficient",
    ascending=False
)

coefficients.head(10)
```

Out[126...

	Feature	Coefficient
7	Inflight entertainment	0.729680
10	On-board service	0.315639
2	Seat comfort	0.288713
13	Checkin service	0.281925
9	Ease of Online booking	0.243490
11	Leg room service	0.236283
15	Online boarding	0.144009
5	Gate location	0.135747
8	Online support	0.105282
12	Baggage handling	0.094965

In [127...

coefficients.tail(10)

Out[127...

	Feature	Coefficient
1	Flight Distance	-0.000182
17	Arrival Delay in Minutes	-0.006918
0	Age	-0.008891
6	Inflight wifi service	-0.089337
4	Food and drink	-0.212719
3	Departure/Arrival time convenient	-0.225801
20	Class_Eco	-0.701087
21	Class_Eco Plus	-0.764463
19	Type of Travel_Personal Travel	-0.781609
18	Customer Type_disloyal Customer	-1.899773

Model Performance Interpretation

The Logistic Regression model achieved an accuracy of **82.74%**, indicating that the model correctly classified passenger satisfaction for most customers.

Performance Metrics

Metric	Value
Accuracy	0.8274
Precision	0.8434
Recall	0.8429

The model demonstrates strong predictive ability, with precision and recall both above 84%, showing that it effectively identifies satisfied passengers while maintaining balanced performance across both classes.

Confusion Matrix Interpretation

The confusion matrix produced the following results:

Actual / Predicted	Unsatisfied	Satisfied
Unsatisfied	9437	2238
Satisfied	2246	12055

Interpretation

- **True Negatives (9437):** Unsatisfied passengers correctly classified.
- **False Positives (2238):** Unsatisfied passengers incorrectly classified as satisfied.
- **False Negatives (2246):** Satisfied passengers incorrectly classified as unsatisfied.
- **True Positives (12055):** Satisfied passengers correctly classified.

The model performs consistently across both classes, with relatively balanced numbers of false positives and false negatives.

Classification Report Interpretation

Class 0 (Neutral or Dissatisfied)

- Precision = 0.81
- Recall = 0.81
- F1-score = 0.81

The model correctly identifies approximately 81% of dissatisfied passengers.

Class 1 (Satisfied)

- Precision = 0.84
- Recall = 0.84
- F1-score = 0.84

The model performs slightly better at identifying satisfied passengers.

Overall Performance

The weighted average F1-score of 0.83 indicates strong overall classification performance.

Feature Importance Interpretation

Logistic Regression coefficients indicate how strongly each variable influences customer satisfaction.

Features Associated with Lower Satisfaction

Feature	Coefficient
Customer Type_disloyal Customer	-1.8998
Type of Travel_Personal Travel	-0.7816

Feature	Coefficient
Class_Eco Plus	-0.7645
Class_Eco	-0.7011
Departure/Arrival Time Convenient	-0.2258
Food and Drink	-0.2127
Inflight Wifi Service	-0.0893
Age	-0.0089
Arrival Delay in Minutes	-0.0069
Flight Distance	-0.0002

Among all factors, being a disloyal customer had the strongest negative impact on passenger satisfaction. Personal travel and economy-class seating were also associated with lower satisfaction levels.

Positive Drivers of Satisfaction

The following variables increased the likelihood of passenger satisfaction:

Feature	Coefficient
Inflight Entertainment	0.7297
On-board Service	0.3156
Seat Comfort	0.2887
Check-in Service	0.2819
Ease of Online Booking	0.2435
Leg Room Service	0.2363
Online Boarding	0.1440
Gate Location	0.1357
Online Support	0.1053
Baggage Handling	0.0950

Inflight entertainment had the strongest positive effect on customer satisfaction. High-quality onboard service, comfortable seating, and efficient check-in procedures also contributed significantly to increasing passenger satisfaction.

Business Recommendations

1. Improve Inflight Entertainment

Inflight entertainment had the strongest positive coefficient in the model. Investing in movies, music, Wi-Fi connectivity, and digital entertainment systems may significantly increase customer satisfaction.

2. Enhance On-board Service

Friendly and responsive cabin crews contribute positively to passenger experiences. Training staff and improving service quality can increase customer retention.

3. Increase Seat Comfort

Passengers place high value on comfortable seating and sufficient leg room. Airlines should consider cabin upgrades and ergonomic seating designs.

4. Optimize Online Services

Ease of online booking, online support, and online boarding positively influence satisfaction. Improving digital platforms and reducing booking complexity can enhance customer experiences.

5. Focus on Customer Loyalty Programs

Disloyal customers showed the largest negative coefficient. Strengthening loyalty programs and providing personalized rewards may help increase retention and satisfaction.

6. Improve Economy-Class Experience

Economy and Economy Plus passengers were less likely to report satisfaction. Enhancing services for these customer segments may improve overall satisfaction rates.

Limitations

- Logistic Regression assumes a linear relationship between predictors and the log-odds of customer satisfaction.
- Customer preferences may change over time, requiring periodic model retraining.
- Correlation among variables may affect coefficient interpretation.
- More sophisticated models such as Random Forest, Gradient Boosting, and XGBoost may achieve higher predictive performance.

Future Work

Future improvements may include:

- Hyperparameter tuning.
- Cross-validation.
- Feature selection techniques.
- Addressing class imbalance.
- Comparing Logistic Regression with Random Forest and XGBoost models.
- Deploying the model in a real-time airline customer analytics system.

Conclusion

A Logistic Regression model was successfully developed to predict airline customer satisfaction. The model achieved an accuracy of 82.74%, with precision and recall values above 84%.

The analysis revealed that inflight entertainment, onboard service, seat comfort, and check-in service are among the strongest drivers of customer satisfaction, while customer disloyalty, personal travel, and economy-class seating negatively influence satisfaction.

These findings provide actionable insights that airlines can use to improve service quality, increase customer loyalty, and enhance overall passenger experiences.